

YOLO Object Recognition and Tracking (No-PTZ)

1. Experiment Objective

Click any object detected by YOLO with the mouse, and the car rotates to keep tracking it.

2. Model and Configuration Check

The YOLO ONNX follow node reads the configuration file by default:

```
~/microros_ws/src/yolo_onnx_follow_pkg/config/yolo_onnx_follow_config.yaml
```

Pay special attention to the following parameters:

```
YOLO_ONNX_FOLLOW:
  model_path: "/home/yahboom/MODELS/yolo/yolo11n.onnx"
  image_topic: "/camera/image_raw"
  input_width: 640
  input_height: 640
  conf_threshold: 0.2
  nms_threshold: 0.45
  max_detections: 3
```

If the model path on your machine is different (**you can replace it with your own trained model**), modify `model_path`, or override it at launch time with `model_path:=`:

```
ros2 launch yolo_onnx_follow_pkg yolo_onnx_follow.launch.py
model_path:=/home/yahboom/MODELS/yolo/yolo11n.onnx
```

Replacing It with a Custom Model: What Needs to Be Changed in the Program

In general, when switching to a standard YOLOv8/YOLOv11 ONNX model, you do not need to modify the Python source code — only the configuration file:

- `model_path`: change it to the path of your own `.onnx` model.
- `class_names_path`: change it to the path of your own class-name file.
- `input_width`, `input_height`: change them to the input size used when exporting the ONNX model.
- `target_class` or `target_class_id`: if you want to automatically follow a certain class of targets, set it here.
- `conf_threshold`: lower it when your self-trained model misses detections; raise it when there are many false detections.

There are mainly two cases where you need to modify the source code.

First case: you want to hardcode the class names directly in the program instead of using `class_names_path`. You can modify `COCO_CLASS_NAMES` at the top of `yolo_onnx_follow_pkg/yolo_onnx_follow_node.py`:

```
COCO_CLASS_NAMES = [  
    "person",  
    "cup",  
    "bottle",  
]
```

Using `cClass_names_path` is recommended, so you do not have to modify the source code when switching models.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

You need to change the vehicle model to the corresponding one in the `.bashrc` file:

```
gedit .bashrc
```

```
source /opt/ros/humble/setup.bash  
source ~/workspaces/MicroROS-V2-ROS2-SRC/microros_car_v2_ws/install/setup.bash  
source ~/uros_ws/install/setup.bash  
export ROS_DOMAIN_ID=66  
export CAR_TYPE=ptz          #ptz | no_ptz | basic_car 三种变量  
#echo "LOCAL_IP: $(hostname -I | awk '{print $1}')"  
echo "===== Current IP ====="  
#ip -o -4 addr show scope global | awk '{split($4,a,"/"); printf "%-10s %s\n", $2":", a[1]}'  
ip -o -4 addr show scope global | awk '{  
    split($4,a,"/");
```

After modifying, save and exit, then run the following command to make it take effect:

```
source ~/.bashrc
```

2. Start the Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

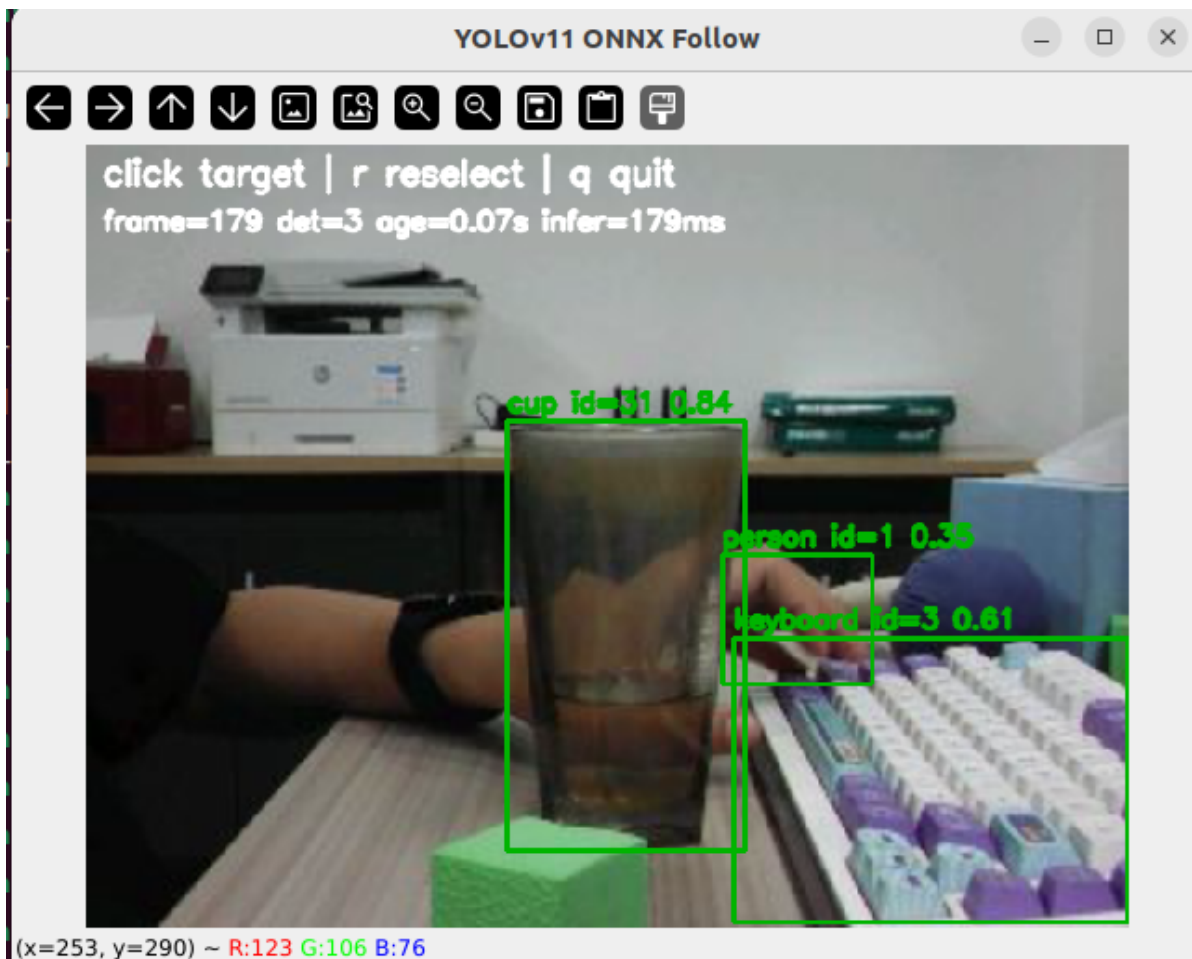
```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050530] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.785741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully

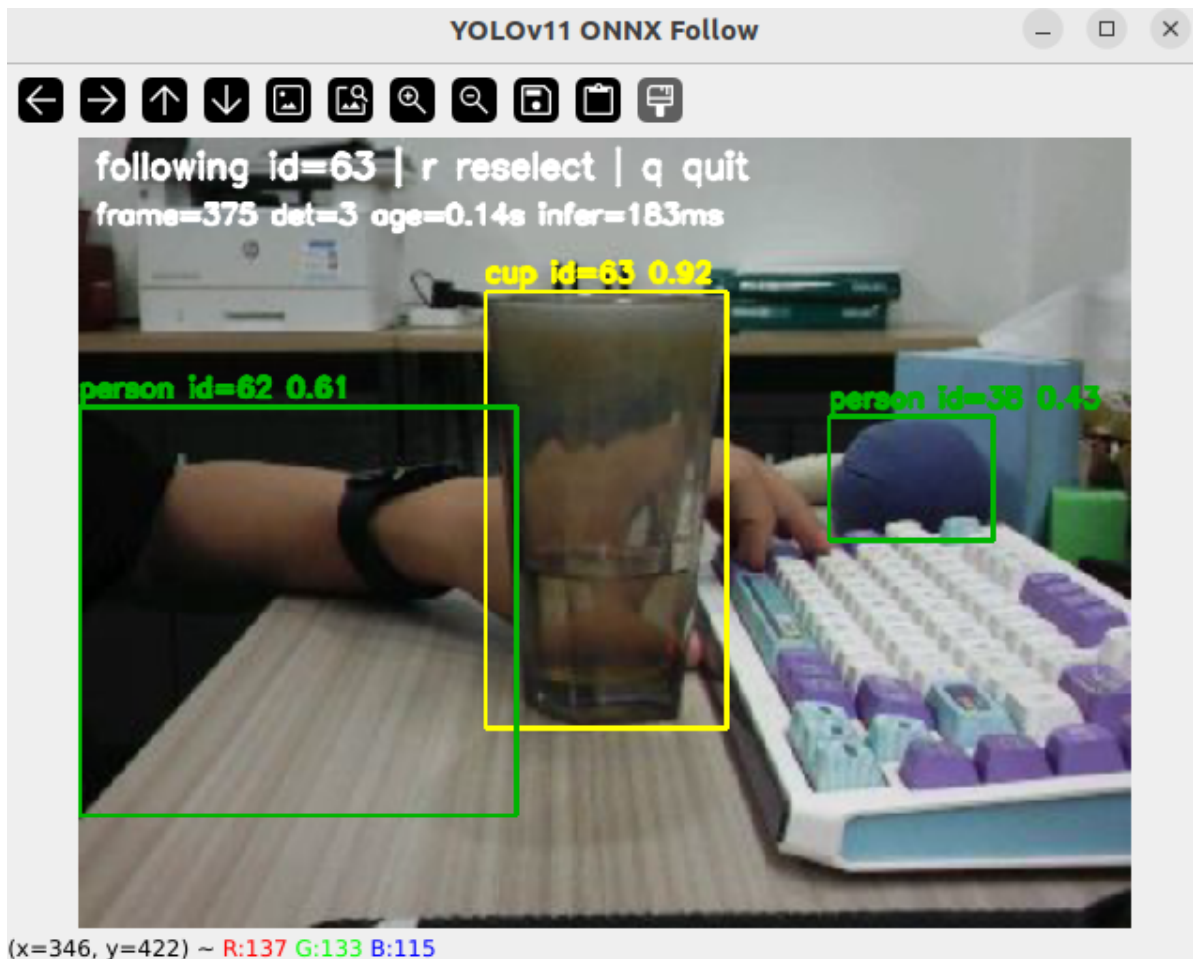
```

3. Start YOLO Object Recognition and Tracking (Terminal 2)

```
ros2 launch yolo_onnx_follow_pkg yolo_onnx_follow.launch.py
```



Click the object to follow with the mouse once it is detected:



Operation Method

After the program starts, an OpenCV window pops up showing the detection boxes, class names, tracking IDs, and confidence scores.

- Left-click a target box: select it and start following that target.
- **r**: clear the current target, stop chassis steering, and re-select a target.
- **q** or **Esc**: exit the program; on exit it publishes a stop velocity and returns the PTZ gimbal to its initial angle.

Normal behavior:

- Green detection boxes appear in the window.
- The target box turns yellow after being clicked.
- When the target deviates from the frame center, the PTZ gimbal rotates first to correct it.
- When the PTZ horizontal angle deviation is large, the chassis publishes `angular.z` to turn in place.
- When the target has been lost for longer than the threshold, the chassis stops steering; if a target of the same class reappears near its previous position within a short time, the program attempts to reacquire it.

At this point, slowly move the detected object, and the PTZ gimbal and the car chassis will track it.

4. Source Code Explanation

4.1 File Structure

```
yolo_onnx_follow_pkg/
├─ launch/
│   └─ yolo_onnx_follow.launch.py
├─ config/
│   └─ yolo_onnx_follow_config.yaml
├─ yolo_onnx_follow_pkg/
│   ├── yolo_onnx_follow_node.py
│   └─ ptz_controller.py
├─ setup.py
└─ package.xml
```

4.2 Configuration File

`config/yolo_onnx_follow_config.yaml` is divided into three main sections:

- **SYSTEM**: configures the chassis velocity topic, default `/cmd_vel`.
- **PTZ**: configures the PTZ angle topic, servo limits, initial angle, publish count, and whether to return to center after stopping.
- **YOLO_ONNX_FOLLOW**: configures the model path, image topic, detection thresholds, window display, target selection, tracking reacquisition, and control parameters.

Commonly tuned parameters:

Parameter	Description
<code>conf_threshold</code>	Detection confidence threshold; the lower it is, the easier detections are, but also the more false detections
<code>max_detections</code>	Maximum number of high-confidence detection boxes kept per frame
<code>max_inference_rate_hz</code>	Maximum inference rate; lower it if performance is insufficient
<code>follow_rate_hz</code>	Follow control rate
<code>pixel_tolerance</code>	When the target center error is smaller than this pixel value, the PTZ is not corrected
<code>kp_x</code> , <code>kd_x</code>	PTZ horizontal control parameters
<code>kp_y</code> , <code>kd_y</code>	PTZ tilt control parameters
<code>max_step_xy</code>	Maximum PTZ angle step per control cycle
<code>enable_body_turn_follow</code>	Whether to allow the chassis to turn following the PTZ horizontal deflection
<code>body_kp</code> , <code>body_kd</code>	Chassis steering control parameters
<code>max_angular_z</code>	Maximum chassis angular velocity

4.3 Core Source Code Explanation

The main program file is:

```
yoIo_onnx_follow_pkg/yoIo_onnx_follow_node.py
```

Its core workflow can be summarized as:

```
Read configuration -> initialize ROS topics -> load ONNX model -> receive camera images
-> YOLO inference -> NMS filtering -> assign tracking IDs -> select/reacquire target
-> compute PTZ angles -> compute chassis angular velocity -> publish control commands
```

4.3.1 Detection Data Structure

`Detection` is used to store the information of each detection box:

```
@dataclass
class Detection:
    bbox: tuple[float, float, float, float]
    score: float
    class_id: int
    class_name: str
    track_id: int = -1
```

Where:

- `bbox`: detection box coordinates in the format `(x1, y1, x2, y2)`.
- `score`: confidence score.
- `class_id`: class ID.
- `class_name`: class name.
- `track_id`: a temporary tracking ID assigned to the target internally by the program.

`cx`, `cy`, `width`, and `height` are property functions used to quickly get the target center point and box size; the subsequent PTZ control mainly uses `cx` and `cy`.

4.3.2 SimpleIoUTracker Tracking ID Assignment

`SimpleIoUTracker` is a lightweight local tracker that does not depend on additional tracking libraries. It ensures that the same object in consecutive frames keeps the same ID as much as possible.

The core logic is in `update()`:

```
tracked = self.tracker.update(
    detections, image_width=frame.shape[1], image_height=frame.shape[0]
)
```

Matching order:

1. First compute the IoU for targets of the same class; when the IoU is greater than `track_iou_threshold`, the old ID is inherited.

2. If IoU matching fails, fall back to matching by the center-point distance of targets of the same class.
3. If it still cannot be matched, assign a new `track_id`.

The purpose is to reduce tracking jitter when objects of the same class move quickly, when detection boxes suddenly grow or shrink, or when IDs temporarily jump.

Related configuration:

```
track_iou_threshold: 0.2
track_center_threshold_ratio: 0.12
track_max_lost_frames: 30
```

The larger `track_center_threshold_ratio` is, the easier it is to merge nearby targets of the same class into the same ID; however, if several same-class targets are very close together in the frame, a value that is too large may cause incorrect matching.

4.3.3 Model Loading `_load_model()`

`_load_model()` is responsible for checking the model path and loading the ONNX model:

```
model_path = str(self.demo_config.get("model_path", "")) or ""
if not model_path or not os.path.exists(model_path):
    raise FileNotFoundError(...)
self.net = cv2.dnn.readNetFromONNX(model_path)
```

If `use_cuda` is `true`, the program tries to use the CUDA backend of OpenCV DNN:

```
self.net.setPreferableBackend(cv2.dnn.DNN_BACKEND_CUDA)
self.net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA_FP16)
```

If the current OpenCV has no CUDA support, keep this in the configuration:

```
use_cuda: false
```

4.3.4 Image Callback `_image_callback()`

After a camera image enters the program, it is first converted to an OpenCV image via `cv_bridge`:

```
frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
```

Then `max_inference_rate_hz` is used to limit the maximum inference rate, avoiding running inference on every frame and saturating the CPU:

```
if now - self.last_inference_time < 1.0 / max_rate:
    return
```

The actual recognition and tracking workflow is:

```

detections = self._run_yolo(frame)
tracked = self.tracker.update(detections, image_width=frame.shape[1],
image_height=frame.shape[0])
self._update_selected_target(tracked, frame.shape[1], frame.shape[0])

```

That is, first YOLO detection, then ID assignment, and finally updating the target currently being followed.

4.3.5 YOLO Inference `_run_yolo()` and Result Parsing `_decode_outputs()`

`_run_yolo()` first letterboxes the original image to the model input size:

```

letterboxed, scale, pad_x, pad_y = self._letterbox(frame, input_w, input_h)

```

Then it constructs a blob and runs the ONNX forward inference:

```

blob = cv2.dnn.blobFromImage(letterboxed, 1.0 / 255.0, (input_w, input_h),
swapRB=True, crop=False)
self.net.setInput(blob)
outputs = self.net.forward()

```

`_decode_outputs()` does four things:

1. Calls `_parse_yolo_row()` to parse each output row.
2. Uses `conf_threshold` to filter out low-confidence results.
3. Uses `cv2.dnn.NMSBoxes()` to remove duplicate boxes.
4. Sorts by confidence and keeps only `max_detections` targets.

The current default configuration is:

```

conf_threshold: 0.2
max_detections: 3

```

Therefore, the window displays at most the 3 targets with the highest confidence, reducing the interference of cluttered detection boxes on selection and tracking.

4.3.6 Target Selection and Reacquisition

There are mainly two entry points for target selection:

- `_on_mouse()`: after clicking a detection box in the window with the mouse, records the target's `track_id` and class.
- `_select_detection()`: finds the target that should currently be followed from the detection results each frame.

After clicking a target, the program saves:

```

self.selected_track_id = int(detection.track_id)
self.selected_class_id = int(detection.class_id)
self.selected_last_bbox = detection.bbox

```


If the same `track_id` can still be found in the next frame, tracking continues directly. If the ID jumps, `_try_reacquire_selected_detection()` is entered to re-find the target based on the same class and the previous frame's position.

The reacquisition score considers three factors:

- The IoU with the previous frame's detection box.
- The distance to the previous frame's target center point.
- The confidence of the current detection box.

Related configuration:

```
reacquire_enable: true
reacquire_iou_threshold: 0.05
reacquire_max_center_ratio: 0.45
reacquire_max_lost_frames: 30
reacquire_score_weight: 0.15
```

`_smooth_target_center()` smooths the target center point:

```
target_center_smoothing_alpha: 0.65
```

The smaller this parameter is, the smoother the target center point is, but the response is slower; the closer to `1.0`, the faster the response, but detection jitter in the frame is more noticeable.

4.3.7 OpenCV Display and Keyboard Input

`_show_frame()` is responsible for drawing detection boxes, class names, IDs, and status information:

- Normal detection boxes are green.
- The currently followed target is yellow.
- The top displays the current frame number, detection count, image latency, and inference time.

`_handle_window_key()` handles keyboard input:

```
if key in (ord("r"), ord("R")):
    self._clear_selection()
elif key in (ord("q"), ord("Q"), 27):
    self.shutdown_requested = True
```

Pressing `r` clears the current target and publishes zero velocity; pressing `q` or `Esc` exits the program.

4.3.8 PTZ Following `_apply_ptz_follow()`

The input of PTZ following is the target center point:

```
err_x = float(state["cx"]) - half_w
err_y = float(state["cy"]) - half_h
```

If the error is smaller than `pixel_tolerance`, no correction is applied to avoid small jitter:

```
if abs(err_x) <= cfg["pixel_tolerance"]:
    step_x = 0.0
```

Otherwise, after normalizing the error, P+D control computes the angle step:

```
out_x = clamp(cfg["ptz_kp"] * norm_x + cfg["ptz_kd"] * deriv_x, -1.0, 1.0)
step_x = out_x * cfg["max_ptz_step"]
```

Finally, the PTZ angle is published through `PtzController.publish_servo_angles()`:

```
self.ptz.publish_servo_angles(
    servo1_angle=...,
    servo2_angle=...,
    repeat=False,
)
```

If the PTZ movement direction is reversed, first change this in the configuration:

```
sign_x: 1.0
sign_y: -1.0
```

4.3.9 Chassis Steering `_compute_body_turn()`

The chassis does not steer directly based on the image pixel error, but based on the PTZ horizontal deflection. In this way, the PTZ first tracks the target, and then the chassis slowly turns the car body back toward the target.

Judgment logic:

```
if abs(servo_err) >= cfg["servo1_engage_deg"]:
    self.body_correcting = True
elif abs(servo_err) <= cfg["body_tolerance_deg"]:
    self.body_correcting = False
```

That is:

- When the PTZ horizontal angle deviation exceeds `servo1_engage_deg`, the chassis starts turning.
- When the PTZ returns within `body_tolerance_deg`, the chassis stops turning.

Chassis angular velocity computation:

```
angular_z = cfg["body_dir"] * (
    cfg["body_kp"] * servo_err + cfg["body_kd"] * deriv
)
angular_z = clamp(angular_z, -cfg["max_angular_z"], cfg["max_angular_z"])
```

If the chassis steering direction is reversed, modify:

```
body_dir: -1.0
```

If the chassis turns too slowly, increase `body_kp` appropriately; if it turns too aggressively, lower `body_kp` or `max_angular_z`.

4.3.10 PTZ Controller `ptz_controller.py`

`ptz_controller.py` encapsulates the PTZ angle publishing logic. The topic it publishes is:

```
/cmd_ptz_angle
```

The message type is:

```
geometry_msgs/msg/Vector3
```

Where:

- `vector3.x`: horizontal servo angle.
- `vector3.y`: tilt servo angle.
- `vector3.z`: currently unused.

`publish_servo_angles()` first clamps the angles with the servo limits, then publishes:

```
self.servo1_angle = clamp(new_servo1_angle, self.servo1_min_angle,  
self.servo1_max_angle)  
self.servo2_angle = clamp(new_servo2_angle, self.servo2_min_angle,  
self.servo2_max_angle)
```

When the program exits, `recenter_after_stop()` returns the PTZ gimbal to its initial angle.